
L01 – Introduction to the Unix/Linux OS

0. Introduction

In academic seismology, one typically has to process hundreds to tens of thousands of files. Because it is research, there is no one set way of doing things, and there is little money or time to develop fancy software to do everything at the press of a button. What this means is that seismologists need to work with a computer operating system that gives them an easy means to do an operation on many files. This means Unix, Linux, or OSX. For example, in Unix it's easy to find every text file over 50 MB that was made in the last 10 days, and to change all of the "r"s to "b"s. In fact, this would take about 3 lines in what we call a shell script. Using the "find" command one could probably do it in one line.

A more relevant example is this. I was asked to join a project working on the Colorado Plateau. My part of the project was to calculate when various types of seismic waves arrive at seismometers at the Colorado Plateau, and I had about a month to complete this task. This involved:

- 1) Making a data request for P, PcP, PP, PKiKP S, ScS, and other phases, for magnitudes > 5.8, with signal-to-noise ratios of > 2.5, for a limited time window around the predicted arrival time of each phase, for a series of seismometers located over a specific geographic area. I used a program called Standing Order for Data (SOD) to do this.
- 2) This returned thousands of files—too much to efficiently look at individually, so I first wrote a Bash script that called a program called Seismic Analysis Code (SAC), to filter out some of the noise.
- 3) Next I noticed that the data request returned some duplicate events, so I wrote a Matlab script that identified duplicate events and removed them.
- 4) I wrote a 2nd Matlab script that looked at each event and quantified how much each seismogram looked like seismograms from nearby stations. This allowed me to identify events that were too noisy or that had other problems.
- 5) I wrote a 3rd Matlab script to loop through all of the events and get the travel times for each phase.
- 6) I wrote a Bash script to compile all of the travel times, and put them into the correct format for some Fortran code that a colleague had to convert the travel times to a model of seismic velocities.

We will spend the first part of the class learning the basics of Unix, Bash scripting, SAC, SOD and other data request tools, and Matlab. With these you can cobble together a way to analyze most seismic data. Some folks also use Perl and Python, and for programming C and Fortran. We won't have time to go into these, but I want you to know these are also in the typical toolbox of a seismologist. One other program that is often used is Generic Mapping Tools (GMT), which is an open source and free program that produces beautiful maps, including most maps you find in seismology and geophysics papers. With the exception of Matlab, all the other software I've described is available for free for Linux and Macs (sometimes getting something to work with a Mac takes a little work), and I encourage you to try downloading and installing these. Windows is great for word processing and making figures, but it's fundamentally unsuited to processing large amounts of data the way we do in seismology. There are acceptable open source ways of word processing and creating figures, but I find it

useful to do these either on a Windows virtual machine running on my linux computer, or on my Mac.

One more example: I was at a meeting recently on the latest work on inversion of seismic data using full waveform physics. We did a couple of workshops; at these workshops every participant was expected to understand how to work from the command line, including how to edit files. We'll learn the basics of that today.

1. What is Unix?

Unix is an operating system (OS): it manages the way the computer works by driving the processor, memory, disk drives, keyboards, video monitors, etc. and by performing useful tasks for the users. **Unix was created in the late 1960s as a multiuser, multitasking system for use by programmers.** The philosophy behind the design of Unix was to provide simple, yet powerful utilities that could be pieced together in a flexible manner to perform a wide variety of tasks.

A key difference between the Unix OS and others you are familiar with (e.g., PC) is that Unix is designed for multiple users. That is multiple users may have multiple tasks running simultaneously. Its original purpose was to facilitate software development. It is the primary OS used by physical scientists everywhere, and all supercomputing facilities use it. To put it bluntly, if you are at all on the numerical side of physical sciences, then you need to learn how to operate on a Unix OS.

In this class we are actually using a Linux OS, Fedora 22. What is Linux? Basically the same thing as Unix. **Only, Linux is developed by user contributions.** Several flavors have arisen (Red Hat, Suse, Fedora, etc.) but they are all basically the same thing. What you can do in Unix you can do in Linux (the corollary of which isn't necessarily true). In this class I may refer to Unix and Linux interchangeably. If I say Unix I usually mean Linux, and most of what I say is applicable to both, but there can be some subtle differences.

The main difference between the two is that: (1) Unix development has corporate support. This means it tends to be a more stable OS and is the choice of those for whom stability is the top priority, (2) Linux is developed by a community of users and is free. Thus, you get what you pay for? Well, it has some stability issues and bugs creep up. But, the bugs are also quickly squashed and new content, programs, and functionality has quickly outpaced that of Unix.

I'm not going to go into detail into what the Unix/Linux OS is comprised of, but there are 3 basic entities:

- 1) **The Kernel** – The core of the UNIX system. Loaded at system start up (boot); manages the entire resources of the system. Examples of what it does are: interpreting and executing instructions from the shell, managing the machine's memory and allocating it to processes, scheduling the work done by the cpu's.
- 2) **The Shell** – Whenever you login to a Unix system you are placed in a shell program. The shell is a command interpreter; it takes each command and passes it to the operating system kernel to be acted upon. It then displays the results of this operation on your screen. Several shells are usually available on any Unix system, each with its own

strengths and weaknesses. Examples are the Bourne Shell (**sh**), C Shell (**csh**), and Bourne Again Shell (**bash**).

- 3) **Utilities** -- UNIX provides several hundred utility programs, often referred to as commands. The commands accomplish universal functions such as printing, editing files, etc.

2. Logging into the Linux side of things

- 1) If you don't have a WCNR account, please go to <http://accounts.wanercnr.colostate.edu> and sign up for a WCNR computer account. With this you can log into the computers in the SAL.

- 2) Log into one of the windows computers. Use the search function to search for "xming" and then run it. It won't do anything really visible, but it will make the windowing software (X11) needed run.

- 3) Now double-click on **putty**.

- a) for a server location, type in geo1.wanercnr.colostate.edu

- b) In the menu, click on "X11", and then check the "SSH Forwarding" box.

You should now have a terminal window you can use.

3. Getting started – really basic Linux

Now that you've logged in and opened up a terminal you are looking at a window that contains your home directory space. In case you are already confused, on Linux systems we refer to **folders** as **directories**.

When you log in, you are automatically put in your home directory. Right now you'll see something like this in the terminal window

```
Geo1 ~ %
```

This is called your prompt, and can be customized to be many different things (google "customize bash prompt" for instance). Right now geo1 refers to the computer on which you are working. The tilde (~) is a shorthand way of saying that your home directory. The percent sign is one way traditionally used to indicate the end of the prompt.

Just for kicks, let's change the prompt. Type:

```
export PS1=">> "
```

You'll now see the prompt is just

```
>>
```

If that doesn't work, type

```
Set prompt = ">>"
```

A couple of notes on your home directory

- You are in control of your home directory and the files that reside there. You are also in control of the file access permissions to the files in your home directory. Generally, you alone should be able to create/delete/modify files in your home directory. Others may have permission to read or execute your files as you determine.
- In most LINUX systems, you can move around or navigate to other parts of the file system outside of your home directory. This depends upon how the file permissions have been set by others and/or the System Administrator.

Linux Commands

Linux commands are programs that are supplied with the Linux OS to do specific tasks. They generally act like:

```
>> command arguments
```

Unlike your PC or Mac, instead of clicking a program icon, you type a program name in the terminal window. Typing a command here is called working with the command line interface (CLI), and is a typical way to work in a Linux environment. For example, type the following:

```
>> pwd
```

You'll see output that looks something like this, except it will be your personal home directory, which will have a 30GB quota.

```
/nfs/fac/schutt
```

pwd means print working directory, which means to show the location of the directory in which you are working. Now we know our home directory, but if you ever forget type

```
>> cd ~
```

which will change directory (cd) to your home directory (~). Let's try another command

```
>> date
```

date is an example of a Linux command. When used as above it simply returns the current date and time. But, we can often supply arguments to the command that modify the way the program works. For example:

```
>> date --date="yesterday"
```

Here we supplied an argument asking us to return yesterday's date instead of today's. Say you want to know what date the homework is due. Type

```
>> date --date="next tuesday"
```

Note that Linux/Unix is case sensitive. So typing **Date** won't work (try it).

One of the most important Linux commands is the **ls** (list) command. It lists the contents of the current directory you are in. Try the following:

```
>> ls
>> ls -l
>> ls -la
```

We can create a new directory with the **mkdir** (make directory) command. Try:

```
>> mkdir garbage
```

Now entering the **ls** command should show us that we now have a new directory called **garbage**.

We can go into this directory by using the **cd** (change directory) command:

```
>> cd garbage
```

To move back out of the garbage directory into the previous directory type:

```
>> cd ../
```

Note that we can go back multiple directories if we want to:

```
>> cd ../../../      (etc.)
```

In this case, the **..** always stands for the previous directory. After moving around directories it can get confusing as to where you are. So you can always use **pwd** to see where you are. You can always go right back to your home directory by typing either:

```
>> cd ~username
```

or just

```
>> cd ~
```

or even just

```
>> cd
```

The primary reason to use the tilde (~) is so that we can back to directories starting from our home directory. (e.g., `>> cd ~/Music` if the **Music** directory was located in my home directory)

Perhaps your sick of your directory called garbage. You can get rid of it with **rmdir** (remove directory):

```
>> rmdir garbage
```

We can also make files. We will talk more about this now, but let's just try the following:

```
>> echo "I love geophysics" > geophys.txt
>> echo "I really love geophysics" >> geophys.txt
```

The **echo** command just echoes whatever you write, and in this case redirected (the `>` and `>>` commands) "I love geophysics" into a text file called **geophys.txt**. Perhaps I wasn't happy with the file name I created and wanted it be named **Geophys.txt** (note that we are working in the Bash Shell and that file names are case sensitive), then I could use the **mv** (move) command:

```
>> mv geophys.txt Geophys.txt
```

Or maybe I wanted another copy of this file **Geo.txt** called **Geo_copy.txt**

```
>> cp Geophys.txt Geo_copy.txt
```

You can guess already that the **cp** command means copy. As you can tell, there are a lot of Linux commands. The examples shown above are some of the most important, but are really just the tip of the iceberg. The following web page shows what some of the most important basic commands are:

<http://www.dummies.com/how-to/content/common-linux-commands.html>

Special Characters

There are also some very special characters that you can type in Linux. The next table shows just a few of them:

Character	Function
*	wildcard for any number of characters in a filename
?	wildcard for a single character in a filename
\$	References a variable
&	executes a command in the background
;	separates commands on the same line
>	redirects standard output

From the above example we should still have two files around named **Geo.txt** and **Geo_copy.txt**. What if I want to see all of the files I have that have a name starting with **Geo**? I can use the ***** character:

```
>> ls Geo*
```

Or just files with the word **copy** in them?

```
>> ls *copy*
```

We will introduce more of these characters later.

Getting Information on Commands

Most Linux commands have several options and can be used in a variety of ways. To get full instructions on a Linux command there is the **man** (manual) utility. For example to see all of the ways you can use the **ls** command type:

```
>> man ls
```

The output of the **man** command can be kind of technical. So, another option that works well in many instances is to simply google “linux ls example”

Logging Off the System

There are a couple of ways to log off a system, and these can vary depending on the type of linux or unix that you are using. For Fedora linux, typically your user name is in the upper right hand part of the screen. In this case our full user name is Virtual. Normally you’d be able to click on this to get a menu that would allow you to log off. In the case of our virtual machine, the only option is to shutdown the virtual machine. A command line way of doing this is

```
>> shutdown -h now
```

If you are remotely logged into a computer, typically you can log out by typing **exit** or **logout** in the command window you used to remotely log into the computer.

4. Editing Files

One of the most important choices you will make in learning Linux is what text editor you should you use. This is likely not a question many of you anticipated, as on a Windows or Mac one rarely ever uses a text editor – unless you refer to Microsoft Word (it is a text editor, but how many times do you store data in **.txt** format?). On a Linux system there are several choices of editors. Peoples defense of their choice of editor is similar to a religious conviction, so be careful in talking bad about other editors. Two of the most popular choices of editors are:

- **vi** – (pronounced *vee – eye*) one of the earliest advanced editors, was installed on every system. *vi or die* was a common expression, as it was the only editor found on many systems. The version on our virtual machine is called **vim**, and typing **vi** will also bring up **vim**.

- **emacs** – a more recent editor, many people prefer this one, and it can now be found as commonly as vi. Emacs has many more commands than vi. I don't really use emacs, but don't be afraid to consider it if you go on to explore linux more.

You can use whatever text editor you choose. The choice is yours, but you are responsible for learning one on your own. Learning to use an editor is not a choice though. This is mandatory if you want to be successful in computation. I have heard not knowing an advanced editor described as “being like a car without an engine under its hood.” I personally like **vi**, which is rather difficult to learn at first. But, I can maneuver around a file in **vi** way faster than any other editor so here's a super fast intro:

To create a new file, or open an old file type:

```
>> vi myfile.txt
```

The key thing to remember is that **vi** has two modes: **command**, and **insert**.

When you are in **command** mode, everything you type on the keyboard gets interpreted by **vi** as a command. Command mode is the mode you start out in.

Now that you are in your file enter into insert mode by hitting the **i** (for **insert**) key. You should notice that at the bottom of the screen it now says you are in **-- INSERT --** mode. Now anything you type shows up on the screen.

When you are in insert mode, you can switch back to command mode by pressing the **Esc** key on your keyboard.

When you are in command mode, there are many keys you can use to get into edit mode, each one gives you a slightly different way of starting to type your text. In addition to insert there is also **a** for **append**, **o** for **open a line**, etc.

To wrap up a **vi** session, hit the **Esc** key to get back into the command mode. Now you save the file by hitting **Shift+ZZ**.

A good tutorial can be found here: <http://www.rru.com/~meo/useful/vi/vi.intro.html>. I have a reference book on it as well.

Let's try creating a file called temp.txt

```
>> vi temp.txt
```

hit the **i** key for insert and type some words, for example here are some nice words from Edward Abbey's famous book Desert Solitaire:

“The love of wilderness is more than a hunger for what is always beyond reach; it is also an expression of loyalty to the earth, the earth which bore us and sustains us, the only home we shall ever know, the only paradise we ever need – if only we had the eyes to see. Original sin,

the true original sin, is the blind destruction for the sake of greed of this natural paradise which lies all around us – if only we were worthy of it.”

Now hit the **Esc** key and save the file by hitting **Shift+ZZ**.

vi has a pretty esoteric command set, but it's very powerful. For instance, you can change all instances of the word “geophysics” in a file with the word “seismology” by typing `%s/geophysics/seismology/g`.

Don't worry if you find vi or emacs to be difficult to learn. They can be. A graphical text editor that I like to use a lot is **gedit**. Type

```
>> gedit temp.txt
```

And play around with the text you added in vi, maybe you can fix any errors in the text too. A nice thing about gedit is that it can recognize many basic types of unix files based on their file extension and color code lines within the text file. When we work on bash scripts you'll see how this is useful. However, having a basic command of vi or emacs is essential for cases when you can only work in a terminal window. This pops up from time to time, either when you are remotely logging into a computer, or during

5. A few more important commands

If we do a **ls** we can see that our file **temp.txt** now exists. But this doesn't tell us anything about what is in the file.

There are several ways we can see the contents of the file. gedit is one, but let's consider some ones that do not bring up new windows. Try the following commands:

```
>> cat temp.txt
```

```
>> less temp.txt
```

So, what was the difference between the two commands?

Now try the following:

```
>> head -1 temp.txt
```

```
>> tail -1 temp.txt
```

These commands are obviously useful if you want to see the top or bottom of a file, say one that *is really large*. What if we want to know something about the file like how many words does it contain? Look up the man page on **wc** (word count) and find out how you can (a) determine how many words the file contains, (b) how many characters the file contains, and (c) how many lines the file contains.

(a)

(b)

(c)

Another really useful command is **grep**. This allows us to search files to find specific instances of words. For example, we could say let's just find the lines in `temp.txt` that contain the word `paradise`.

```
>> grep paradise temp.txt
```

grep is really useful when I'm searching for something specific in a lot of files.

Zippping and Unzipping Files:

There are, as is typical, many choices of zipping utilities. Generally, we use the utility called **gzip**. To zip up, or compress, our file `temp.txt` simply type:

```
>> gzip temp.txt
```

Now if you do an **ls** you will see the filename is changed to `temp.txt.gz`. Note that the `.gz` extension will often be found on files you get from me. This means they have been compressed with **gzip**. What happens if you try and view the contents of this file with `cat temp.txt.gz`?

Right, to view its contents we need to unzip or uncompress it:

```
>> gunzip temp.txt.gz
```

You will also notice that many files you download have the `.tar` extension. These tar files stand for **tape archiving** (still in use for backups today!). Usually we use tar to lump a group of files together into one single file. Then we only need to send one file and not a bunch. To see how tar works let's do as follows:

```
>> cp temp.txt temp_copy.txt
```

```
>> mkdir TempFiles
```

```
>> mv temp*.txt TempFiles
```

```
>> tar cvf TempFiles.tar TempFiles
```

Now you will notice there is a file called `TempFiles.tar`. The **tar** command we used in the above example used the flags `to create a file called TempFiles.tar` from the directory and its contents `TempFiles`.

Most of the files you download from the webpage will have the `.tar` extension. To unpack these files:

```
>> tar xvf TempFiles.tar
```

In this case we used the **extract** flag (x).

A gzipped tar file is often used to send a group of files (e.g. TempFiles.tar.gz). This is colloquially called a “tarball”, as in “send me a tarball of your seismology class.”

Job status:

It is also useful to see what is currently running on the computer you are using. The quickest way to do this is to use the **top** utility. Just type:

```
>> top
```

But, you can get specific information using the **ps** (processes) utility. As an example, to see what programs you personally are using type:

```
>> ps -u username
```

(where you fill in **username** with your personal username). Don’t know your username? Type:

```
>> whoami
```

top and **ps** are especially useful if you’ve started a bunch of jobs or maybe someone else did on your computer and its eating up the cpu or memory. Notice that all jobs have a number associated with them under the column **PID** (Process ID number). This number is important. **Don’t** actually do this now – but in the event that you absolutely need to stop something that is running you can do this with the **kill** command:

```
>> kill -9 PID
```

This will force the process, whatever it is to be stopped. So only use this if you absolutely need to stop the job and you know what the job is.

6. Customizing your environment – the .bashrc and .bash_profile files

We’ve been talking about Linux commands. That’s not strictly true. There’s an interface that takes the commands we type on the command line interface in the terminal, and interprets them for the Linux kernel. This is called the “shell”, and there are different types of shells. They each have a set of common commands: ls, cp, mkdir, etc., but these vary in certain features like how to reference past commands (command line history), and file redirection.

Linux distributions and Mac OSX use Bash (Bourne again shell: [http://en.wikipedia.org/wiki/Bash_\(Unix_shell\)](http://en.wikipedia.org/wiki/Bash_(Unix_shell))) for a shell. A lot of seismologists use C-shell or Tcsh as their shell. I think this is because many of us grew up using Sun

computers running Solaris. There are other shells as well; Wikipedia has a nice summary of them (http://en.wikipedia.org/wiki/Unix_shell). To see what type of shell you have on a given Linux machine, type:

```
>> echo $SHELL
```

I used to use a version of C-shell called Tcsh, but I've found Bash has more features and is more flexible, so I've converted to that. I'm still learning it however!

In each shell there are a couple of files that get executed. Every time you log in to something, either when you log in to your virtual machine, or at a regular Linux computer, or if you remotely log in to a computer through a terminal window, you are using a *login shell*. If you are already logged in to the computer and open an additional terminal, you are using a *non-login shell*. In Bash, every time you start a login shell, the `.bash_profile` file gets executed. When you start a non-login shell, the `.bashrc` file is executed. Why the difference? Say there's something you want to do only when you log into the computer, like list who else is on the computer. You'd put the `who` command in your `.bash_profile`, but not in your `.bashrc`. Stuff you want to do or set each time you bring up a new window you should put in your `.bashrc` file. A good example of this is if you want to customize your prompt from the system default (which we did above).

Under most conditions I can think of, when you run your `.bash_profile` file, you also want to run your `.bashrc` file, so I put this in my `.bash_profile`:

```
if [ -f ~/.bashrc ]; then
    source ~/.bashrc
fi
```

What's this? It's a simple series of bash commands, similar to the bash scripts we will learn to write shortly.

The first line says: "if the file `~/.bashrc` exists then...

The second line says "source (run) `~/.bashrc`".

The third line ends the if statement.

The `.bashrc` and `.bash_profile` files live in your home directory, so change directories to your home directory and let's look at their contents:

```
>> cd ~
```

```
>> cat .bash_profile
```

```
>> cat .bashrc
```

There are two main things I want to point out in this file: (1) your search path, and (2) aliases.

Search Path

When you type a Linux command at the command prompt (e.g., `cd` or `ls`) the Linux Shell looks for a program with that name. Things like `ls` or `cd` or `mkdir` are all programs that reside in a directory somewhere. For example, if you want to find out where the `ls` command lives type:

```
>> which ls
```

So, on the computer I am working on as I write this document, I see that the program `ls` is located at: `/usr/bin/ls` and that `ls --color=auto` is aliased to "`ls`". This means when I type `ls`, the computer automatically interprets it as `ls --color==auto`

Let's discuss the path first. For the Linux system to be able to execute the `ls` command it has to be in the Linux Search Path. That is, Linux has a special variable called `$PATH` that contains a collection of directory names to search through for commands that are typed. To see what directories Linux is currently searching through for you type:

```
>> echo $PATH
```

When you cat `.bash_profile`, you can see that the path is set with this line:

```
PATH=$PATH:$HOME:/local/bin:$HOME/bin
```

OK, but what if I make a program that I want Linux to be able to use (and you probably will)? Or what if I install a program in a different directory than one that's in the path. In these cases we'll want to customize the path. For instance, we may wish to install new programs in our class directory

So, let's modify the path to add the directories where these programs live. Fire up `vi` or `edlin` or `gedit` and modify `.bash_profile` so that

```
PATH=$PATH:/data/class-data/Fall-2016/GEOL578
```

After you've saved the file, we need to tell the Linux system to re-read our `.bash_profile` file. We do this by typing:

```
>> source .bash_profile
```

If you want to run a program infrequently, and see no reason to put it into `PATH`, just type the full path to the file. For instance, to run `sac`, type:

```
>> /opt/sac/bin/sac
```

For what it's worth, `bin` refers to a binary file, and this is the directory in which executable programs are stored.

Aliases

Another fine use of the **.bash_profile** or **.bashrc** file is to create aliases, or shortcuts. For example, instead of just the normal **ls** command you might add the following flags: **ls -F -h --color=always**. So, I can add a line to my **.bashrc** file that says every time I type **ls**, actually do: **ls -F -h --color=always**. We can do this by adding the following line to our **.bashrc** file:

```
alias ls="ls -F -h --color=always"
```

Or you could just type "net" to launch an internet browser:

```
alias net="firefox &"
```

If you don't like an alias, say for **ls**, type

```
unalias ls
```

There are lots of clever examples of aliases on the web.

Do I put something in .bash_profile or .bashrc?

Typically people change **PATH** in **.bash_profile** and add aliases or any functions in **.bashrc**.

7. Homework

Use **vi** or **emacs** and create some files. Create one file and tell me: (1) what year you are in your studies (Ph.D. student, M.S. student, Senior, Junior, etc.), (2) if you are a graduate student tell me who you are working with and what your research project is about or if you are an undergraduate tell me what your plans are after graduation, and (3) what you want to get out of this class. Also, if there is a special computational task or tool you want to learn in this class that isn't currently on the syllabus please tell me what it is and why its important for you to learn that.

This may be helpful: <http://www.lagmonster.org/docs/vi.html>

or this:

http://www.math.ntu.edu.tw/~wwang/mtxcomp2010/cssd2011/download/vi_cheat%20sheet.pdf

or this: <http://www.smashingmagazine.com/2010/05/03/vi-editor-linux-terminal-cheat-sheet-pdf/>

Create a second file and tell me what math and physics classes you have had, if any, beyond Calculus and Physics II.

Now, use **gedit** or **vi** to customize your command prompt in your **.bashrc** file.

Now make a directory and copy these three files into that directory. **tar** the files, and then **gzip** the tar file.

Use the following naming convention:

LastName_Firstname_HW1.tar.gz.

To share materials, we will have a class directory on the WCNR network. Its location is:

`/data/class-data/Fall-2016/GEOL578/AssignmentDropbox`

I will ask you to copy your file to `/data/class-data/Fall-2016/GEOL578/AssignmentDropbox` using a function call `scp`. `scp` and `rsync` are great programs that transfer files. I particularly love `rsync`, because it only transfers files that are not already in the receiving directory. Like most things in linux, it's easy to find examples on the web. Here are two pages that show nice examples for `rsync` and `scp`.

<http://rsync.samba.org/examples.html>

http://www.hypexr.org/linux_scp_help.php

And, here's how to transfer your homework tar file.

```
scp Lastname_Firstname_HW1.tar.gz  
youreID@teewinot.warnercnr.colostate.edu:/data/class-data/Fall-  
2016/GEOL578/AssignmentDropbox
```

That's all one line, above.

Or, if you are on the WCNR linux network, all you have to do is type

```
cp /path/to/Lastname_Firstname_HW1.tar.gz /data/class-data/Fall-  
2016/GEOL578/AssignmentDropbox
```

You're done!

8.0 Acknowledgements

This lab is based on an excellent series of computational geophysics write-ups by Michael Thorne of the University of Utah. He's got really cool animations on his website too. <http://web.utah.edu/thorne/>